Quantix

CANDIDATE ENGINEERING CHALLENGE

Laravel Internship Take-Home Assignment

Service Request & Approval API for a Business SaaS Platform

2 DAYS
Submission window

~15 HOURS
Expected effort

GIT REPO
Source code + README

TECH STACK

Laravel 11+ · PHP 8.2+ · MySQL / PostgreSQL / SQLite · Laravel Sanctum · Pest or PHPUnit · Git

This assignment is designed to understand how you design, build and test a real backend feature, structure your code and explain your technical decisions.

01 / ASSIGNMENT BRIEF

The engineering challenge

Build the backend for a business SaaS module that allows employees to submit internal service requests and enables the responsible teams to review, approve, assign and complete them. Your API will power a web or mobile client built by another team.

SCENARIO

A growing company handles IT support, equipment, maintenance and access requests through messages and spreadsheets. Ownership is unclear, updates are missed and nobody can see the history of a request. The new service should give every request a clear status, an owner and a complete activity history, and it must enforce who is allowed to do what.

YOUR GOAL

- Expose a secure, well-documented REST API for submitting and tracking requests.
- Enforce the approval workflow with explicit rules, not scattered if-statements.
- Notify the right people asynchronously when a request changes state.
- Prove it works with automated tests and a README that anyone can follow in minutes.

IMPORTANT

You are not expected to build a full product or a front end. We value a smaller, well-structured and well-tested solution over a large one that is half-finished. Where the brief is silent, make a reasonable decision and record it in your README.

02 / ROLES & WORKFLOW

Build for three roles

Employee

Creates a request, adds details, responds to returned requests and tracks progress.

Manager

Reviews requests from their team and approves, rejects or returns them with a comment.

Operations User

Receives approved requests, assigns an owner, updates status and records completion.

CORE JOURNEY

1. **Create request**
2. **Manager review**
3. **Approve / reject / return**
4. **Assign owner**
5. **Work in progress**
6. **Complete & close**

BUSINESS RULES TO ENFORCE

- Requests move only through valid status transitions. Anything else returns a 422 with a clear message.
- Employees can edit a request only while it is in Draft or Returned status.
- A manager can act only on requests from their own direct reports.
- Only Operations users can assign, start and complete work.
- Rejected and Completed requests are final and cannot be changed.
- Every transition is recorded with who made it, when, and an optional comment.

03 / TECHNICAL SCOPE

Build it like a real product

Use Laravel 11 or later with PHP 8.2+ and MySQL, PostgreSQL or SQLite. Cover each area below at a depth that fits the time you have.

Data model

Design migrations, relationships, indexes and factories for users, requests, comments and activity.

Authentication & roles

Token authentication with Laravel Sanctum, and Policies or Gates guarding every action.

Workflow engine

Model the status lifecycle in one place, such as a state class or service, with guarded transitions.

Validation & errors

Form Requests, API Resources and consistent JSON errors with correct status codes.

Queues & notifications

Send email or database notifications through queued jobs when a request changes state.

Audit trail

Record every change in an activity log that can be queried per request.

Testing

Feature tests for the API and unit tests for workflow rules, using Pest or PHPUnit.

Performance & quality

Avoid N+1 queries, paginate lists, and format code with Pint plus a static analysis tool.

04 / API & TESTING

Endpoints to deliver

Treat this as the minimum contract. You may add endpoints or change paths if you explain why in your README.

METHOD	ENDPOINT	ROLE	PURPOSE
POST	`/api/login`	Any	Issue an access token
POST	`/api/requests`	Employee	Create a request as draft or submit it
GET	`/api/requests`	All	List with search, status filter, sorting and pagination, scoped to the role
GET	`/api/requests/{id}`	All (policy)	Details with comments and activity history
PUT	`/api/requests/{id}`	Employee	Edit a draft or returned request
POST	`/api/requests/{id}/submit`	Employee	Send for manager review
POST	`/api/requests/{id}/decision`	Manager	Approve, reject or return with a comment
POST	`/api/requests/{id}/assign`	Operations	Assign an owner
POST	`/api/requests/{id}/status`	Operations	Move to in progress or completed
POST	`/api/requests/{id}/comments`	Participants	Add a comment

SEED DATA

Provide seeders so a reviewer can try the API immediately: at least one user for each role, a manager with two direct reports, and about 10 requests spread across all statuses.

RESPONSE STANDARDS

- Return JSON for every response, including errors, with a consistent shape.
- Use correct status codes: 401, 403, 404, and 422 for validation or an invalid transition.
- Paginate list endpoints and return pagination metadata.

TESTING EXPECTATIONS

- A passing happy path for each role, from creating a request to completing it.
- At least one test for each forbidden action that must return 403.
- Tests for invalid status transitions returning 422.
- A test proving that a notification job is dispatched on a status change.

OPTIONAL STRETCH GOALS

File attachments on requests	**Rate limiting on the API**
SLA due dates and overdue flag	**Docker or Laravel Sail setup**

05 / FINAL DELIVERABLES

What to submit

- A Git repository (GitHub or GitLab) with the complete source code and a meaningful commit history.
- A README with setup steps, how to run the tests, seeded login details, your assumptions and what you would do next.
- Migrations and seeders that work with a single php artisan migrate:fresh --seed.
- API documentation: a Postman collection, an OpenAPI file or generated docs.
- An automated test suite that passes with one command.
- An .env.example file and, if you used one, a Docker or Sail setup.
- A short architecture note (half a page) explaining your key decisions and trade-offs.
- A short note listing references, packages and any AI tools used.

SUGGESTED REPOSITORY STRUCTURE

01 - README.md	**02 - docs/ARCHITECTURE.md**
03 - API docs (Postman / OpenAPI)	**04 - database/ (migrations, seeders)**
05 - app/ (Models, Policies, Services)	**06 - tests/ (Feature and Unit)**
07 - .env.example & Docker (optional)	

AI USAGE

You may use AI tools during the assignment. Your submission must still demonstrate your own engineering judgement, and you should be able to explain every part of it. Briefly state where and how AI was used.

06 / SUBMISSION & EVALUATION

Submit within 2 days

Reply to the assignment email with your repository link (invite our reviewer if it is private) and a Google Drive or WeTransfer link to a zip of the project without the vendor and node_modules folders. Please make sure all links are accessible before sending.

EVALUATION CRITERIA



WHAT WE ARE LOOKING FOR

We are interested in how you structure a real feature, protect it with rules and tests, and explain your decisions. A smaller, well-tested solution is stronger than a large one that is half-finished. Shortlisted candidates may be invited to walk us through their solution.

Thank you, and we look forward to reviewing your work.

Quantix Global | info@quantixglobal.com | Pelawatta, Sri Lanka